

Assignment 1 - Rocket Launch

Alec Stanley, ID: 33963975

10/10/2025

Contents

1	Introduction	1
2	Method	1
2.1	Diagrams	1
2.2	Acceleration	2
2.3	Defining forces	3
2.4	Setting up ODEs	4
2.5	Technical notes	4
3	Results	4
3.1	Plots	4
3.2	Values	6
4	Conclusion and Discussion	6
5	Appendix: Code	8

1 Introduction

The task was to create a program to simulate a single-stage rocket launch using the principles of particle dynamics. A series of important data and initial conditions were given, and hand calculations were completed to construct a set of first order linear ordinary differential equations that could be numerically integrated in Python. The rocket's trajectory was then plotted and analysed.

Some important assumptions were made, namely that the Earth is flat and the thrust acts in the direction of motion. The rocket was also modeled as a point mass with linearly decreasing mass. The rocket's trajectory was found to follow an expected path given the assumptions made. The rocket reached a peak altitude of 396.22km, and hit the ground after 734.97 seconds, 3210.72km downrange. The maximum speed was 6.00km/s, and the maximum drag (Max-Q) was 156.63kN.

2 Method

2.1 Diagrams

We can begin with a local coordinate system attached to the rocket, which will be approximated as a point mass. These are normal-tangential coordinates ($\hat{n}, \hat{t}$). The forces involved are thrust, drag, and weight. Thrust acts in the direction of travel, while drag acts opposite to the direction of velocity. Weight acts vertically downwards, where the rocket's angle to the horizon is denoted ϕ . The free body diagram and kinetic diagram are shown in Figure 1.

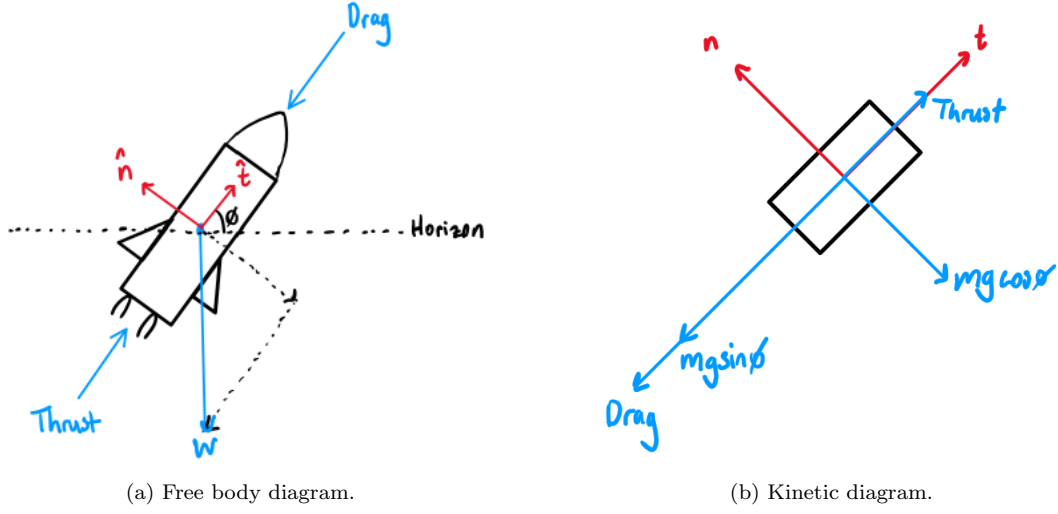


Figure 1: Diagrams in local coordinate system.

2.2 Acceleration

This step involves determining the acceleration of the rocket in Cartesian (x-z) coordinates (a global coordinate system). To begin, we will represent our forces in normal-tangential coordinates, and then transform this to Cartesian.

We will assume the Earth's surface is flat for the purposes of simplicity. Our forces acting are:

$$\begin{aligned} \vec{F}_T &= T \hat{u}_t, \\ \vec{F}_D &= -D \hat{u}_t, \\ \vec{W}_t &= -mg \sin \phi \hat{u}_t, \\ \vec{W}_n &= -mg \cos \phi \hat{u}_n. \end{aligned}$$

We can take the sum of forces in each direction to be

$$\begin{aligned} \sum F_t &= T - D - mg \sin \phi, \\ \sum F_n &= -mg \cos \phi. \end{aligned}$$

When converting to our global coordinate system, where x is downrange distance and z is altitude, we can apply the following transformations:

$$\langle \hat{u}_t, \hat{u}_n \rangle = \langle \cos \phi \hat{u}_x + \sin \phi \hat{u}_z, -\sin \phi \hat{u}_x + \cos \phi \hat{u}_z \rangle.$$

Hence,

$$\begin{aligned} \vec{F} &= (T - D - mg \sin \phi)(\cos \phi \hat{u}_x + \sin \phi \hat{u}_z) - mg \cos \phi(-\sin \phi \hat{u}_x + \cos \phi \hat{u}_z), \\ &= (T - D - mg \sin \phi) \cos \phi \hat{u}_x + (T - D - mg \sin \phi) \sin \phi \hat{u}_z + mg \cos \phi \sin \phi \hat{u}_x - mg \cos^2 \phi \hat{u}_z. \end{aligned}$$

Equating components,

$$\begin{aligned} F_x &= (T - D) \cos \phi - mg \sin \phi \cos \phi + mg \sin \phi \cos \phi \\ &= (T - D) \cos \phi, \end{aligned}$$

$$\begin{aligned} F_z &= (T - D) \sin \phi - mg \sin^2 \phi - mg \cos^2 \phi \\ &= (T - D) \sin \phi - mg. \end{aligned}$$

To determine the acceleration, we need to start with Newton's second law and the definition of force as the time rate of change of momentum.

$$\sum \vec{F} = \frac{d(m(t)\vec{v})}{dt} = \dot{m}\vec{v}_{\text{exhaust}} + m(t)\vec{a}$$

$$\rightarrow m(t)\vec{a} = \sum \vec{F} + \vec{T}, \text{ where } \vec{T} \text{ is the thrust term } = \dot{m}\vec{v}_{\text{exhaust}}.$$

The exhaust velocity acts in the negative direction of travel, meaning the induced thrust is positive in the direction of travel. Note we also have time-varying mass. Since we are assuming thrust is an external force acting on the rocket, we can absorb the thrust term into the sum of forces term. Now we can write the acceleration in each direction by using our more simplified version of Newton's second law:

$$m(t)\vec{a} = \sum \vec{F}, \text{ where } m(t) = m_0 - \dot{m}t.$$

As a result, we can write

$$a_x = \frac{(T - D) \cos \phi}{m(t)},$$

$$a_z = \frac{(T - D) \sin \phi}{m(t)} - g.$$

2.3 Defining forces

Our set of initial conditions are:

Rocket data		Environmental constants	
Rocket diameter d	2.00 m	Earth radius R_0	$R_0 = 6371$ km
Drag coefficient C_D	0.75	Surface gravity g_0	$g_0 = 9.807$ m/s ²
Take-off mass m_0	$m_0 = 5.00 \times 10^4$ kg	Surface density ρ_0	$\rho_0 = 1.21$ kg/m ³
Rocket motor specs		Attitude data	
Thrust T	1.00 MN	Initial launch angle	90° [t < 10 s]
Specific impulse I_{sp}	350 s	Pitch-over angle	70° [10 < t < 20 s]
Burn time	150 s	Final angle	variable [t > 20 s]

Table 1: Rocket and environmental data.

The task is to write our forces of thrust, drag, and weight in terms of these values. We are told we may approximate density as $\rho(z) = \rho_0 e^{az^b}$ where $a = -2.5 \cdot 10^{-5}$ and $b = 1.15$. We may also use the inverse square law to define gravity as $g(z) = g_0(\frac{R_0}{R_0+z})^2$. Hence,

$$T(t) = 1 \cdot 10^6 N \text{ (zero after burn time),}$$

$$D(\rho, v) = \frac{1}{2}\rho(z)C_D A v^2, \text{ where } A = \pi(\frac{d}{2})^2,$$

$$W(t, z) = m(t)g(z), \text{ where } m(t) = m_0 - \dot{m}t \text{ and } \dot{m} = \frac{T}{I_{sp}g_0}.$$

We can also define angle ϕ as a piecewise function:

$$\phi(t) = \begin{cases} 90^\circ, & t < 10s \\ 70^\circ, & 10s \leq t < 20s \\ \arctan(\frac{v_z}{v_x}), & t \geq 20s \end{cases}$$

2.4 Setting up ODEs

The solver we will use requires differential equations of the form $\frac{dy}{dt} = f(t, y)$. The expressions $a_x = \frac{d^2x}{dt^2}$ and $a_z = \frac{d^2z}{dt^2}$ can be written as two sets of two coupled first order ODEs:

$$\frac{dx}{dt} = v_x,$$

$$\frac{dv_x}{dt} = a_x = \frac{(T - D) \cos \phi}{m(t)},$$

$$\frac{dz}{dt} = v_z,$$

$$\frac{dv_z}{dt} = a_z = \frac{(T - D) \sin \phi}{m(t)} - g.$$

We may pass our integrator a state vector $\vec{q}$, which returns $\dot{\vec{q}}$ if $\vec{q}$ is defined as

$$\vec{q} = \begin{bmatrix} x \\ z \\ v_x \\ v_z \end{bmatrix}, \text{ and hence } \dot{\vec{q}} = \begin{bmatrix} v_x \\ v_z \\ a_x \\ a_z \end{bmatrix}.$$

2.5 Technical notes

The simulation was performed in Python using scipy's solve_ivp function with the RK45 method. As required, a timespan of 0 to 1800 seconds (30 minutes) was used along with a chosen time step of 0.01 seconds. If the rocket hits the ground within these 30 minutes, then the simulation will terminate a minute early and plot the results up to that point. The initial state vector was $\vec{q}_0 = [0, 0, 0, 0]$.

Additionally, the drag always acts opposite to the velocity vector, so while thrust acted along ϕ , another angle θ was introduced in the code which was always equal to ϕ except in the first 20 seconds, where ϕ would be locked to the initial values and θ always represented the direction of motion.

3 Results

3.1 Plots

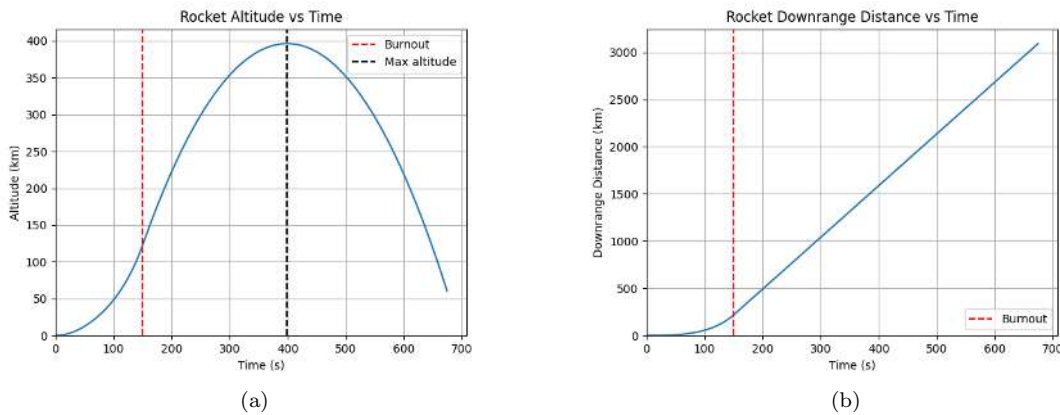


Figure 2: Altitude and downrange distance in km against time (s).

Figure 2 gives us a clear representation of the rocket's trajectory in the x-z plane. We can see the rocket reaches a peak altitude of 396km, and hits the ground after 735 seconds, 3211km downrange. We can see an inflection point in altitude occurs at burnout once thrust stops, and the rocket continues to coast and eventually fall under the influence of gravity. The downrange distance continues to increase at a constant rate after burnout. The results are consistent with expectations given the assumptions made (primarily the flat Earth approximation).

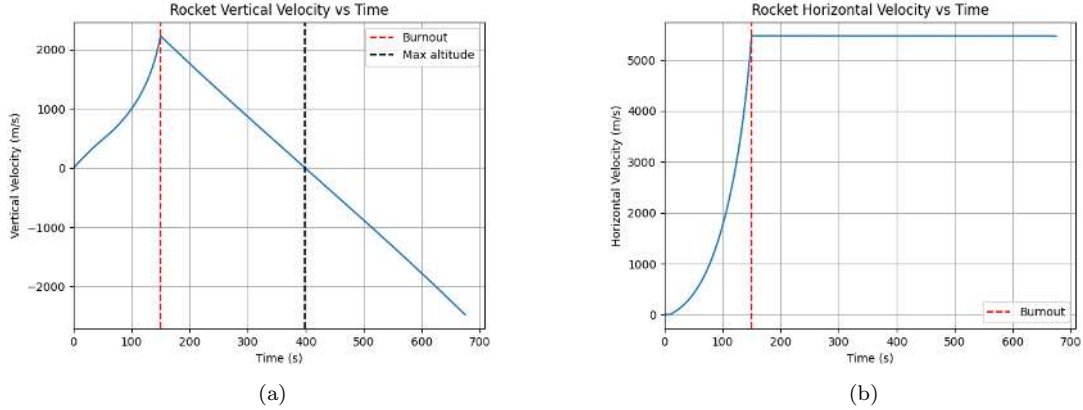


Figure 3: Vertical and horizontal velocity in m/s against time (s).

In Figure 3, we see both vertical and horizontal acceleration due to thrust during the burn phase. After this, the vertical velocity decreases due to gravity, eventually turning the rocket around towards the surface. The horizontal velocity remains constant after burnout as there are no more forces acting in this direction, as the drag is very small at high altitudes.

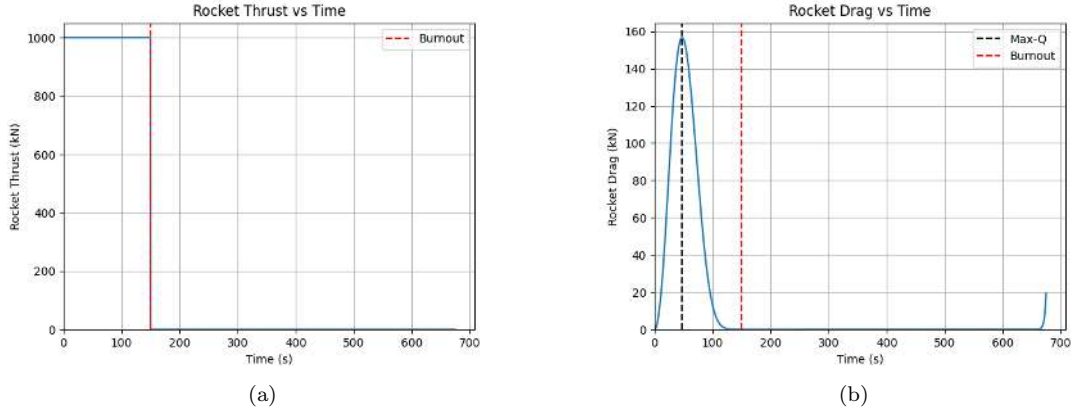


Figure 4: Thrust and drag (kN) against time (s).

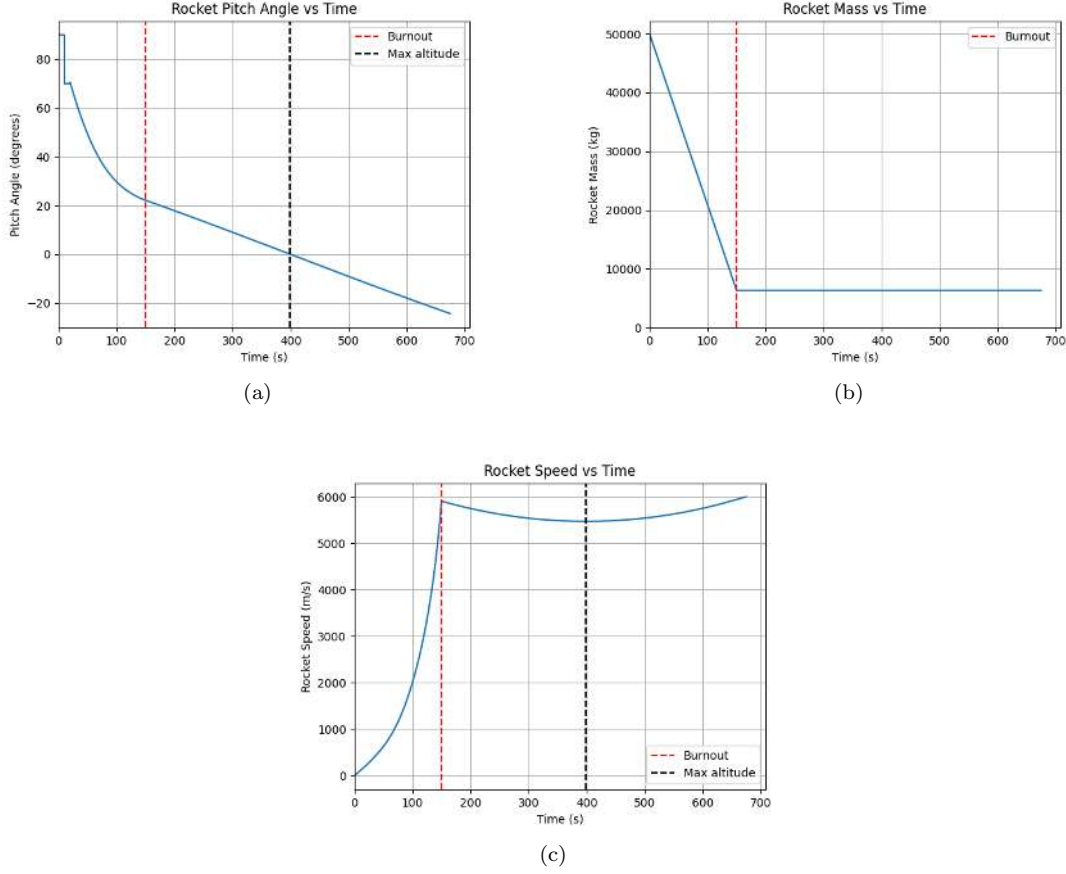


Figure 5: Pitch angle (degrees), mass (kg), and speed (m/s) against time (s).

3.2 Values

Maximum Drag (Max-Q) = 156.63kN, at $t = 47.49$ s,

Maximum Altitude (Apogee) = 396.22km, at $t = 399.32$ s,

Maximum Speed = 6.00km/s, at $t = 674.97$ s,

Impact at ground = 3210.72km downrange, at $t = 734.97$ s.

The simulation itself produced arrays of x, z values of the rocket at each point in time, as well as its horizontal and vertical velocity at these points. This was used to calculate maximum altitude and maximum speed, as well as the final impact point. To find the maximum drag, the resulting velocities were passed into the drag function to get the final array of drag values at every point. The maximum of these is the Max-Q point.

4 Conclusion and Discussion

A main point of note is the approximation that thrust always acts along the direction of motion. In a realistic case, this would not always be true. The rocket is highly likely to be pointing at a greater angle to the horizon than its velocity, especially in the pitch-over phase. As a result, setting ϕ to be locked to the initial values 90° and 70° and then instantly switching to follow the velocity vector causes

an abrupt discontinuity in the rocket's pitch angle at 10 and 20 seconds. To rectify this, it would be more beneficial to define a new angle for the thrust, and have the angles follow smooth curvatures in time rather than step functions. This would be needed for a more realistic simulation and to explore any other effects on the rocket accurately.

In addition, our assumption that the Earth is flat is a large simplification. Our rocket reached altitudes of hundreds of kilometres, and downrange distances of thousands of kilometres. At these scales, the curvature of the Earth would be significant enough that this model would be very far from reality. A more complex model for gravity is necessary, and the resulting trajectory may exhibit more curvature as a result.

Some other areas of inaccuracy lie in the assumption that the engine has constant thrust right until burnout. In reality, the thrust curve should be nonlinear, and it is typical practice to throttle down the engine before cutoff to reduce stress. This would also result in a nonlinear mass flow rate. Density approximations could also be made more accurate by passing in a table of standard atmosphere values instead, and interpolating between them. The drag coefficient would also be variable as it would change with exhaust speed (if variable) and density.

Cutting off our data one minute before collision with the surface was also important, as re-entry causes the rocket to experience extremely high drag forces. In reality, effects at this point become almost ballistic and very difficult to model accurately.

The results produced by this simulation are consistent with expectations, given the assumptions made. There is clearly much more exploration to be done to model a more accurate launch. Overall, this was a successful implementation of particle dynamics principles to model a basic rocket launch.

5 Appendix: Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# Rocket data
DIAMETER = 2 # metres
C_DRAG = 0.75 # Drag coefficient, dimensionless
M_0 = 5 * 10**4 # kg

# Environmental constants
R_EARTH = 6371000 # metres
G_0 = 9.807 # m/s^2
RHO_0 = 1.21 # kg/m^3

# Rocket motor specs
ENGINE_THRUST = 1 * 10**6 # Newtons
SPECIFIC_IMPULSE = 350 # seconds
BURN_TIME = 150 # seconds

# Attitude data
ANGLE_INITIAL = 90 # degrees, t < 10s
PITCH_OVER_ANGLE = 70 # 10 < t < 20s, variable beyond 20s

# Simulation parameters
SIMULATION_TIME = 1800 # seconds
TIME_STEP = 0.01 # seconds

# Initial conditions
x0 = 0 # metres
z0 = 0 # metres
vx0 = 0 # m/s
vz0 = 0 # m/s

state0 = [x0, z0, vx0, vz0] # Pack into initial state vector

def density(altitude):
    # Function returns the air density at a given altitude in kg/m^3

    # For this to work, altitude must be positive
    # The integrator should stop before altitude goes negative, but this is here as a safeguard
    if altitude < 0:
        altitude = 0

    # Empirical formula for air density
    rho = RHO_0 * np.exp(-2.5*10**(-5) * altitude**1.15)

    return rho

def gravity(altitude):
    # Function returns the gravitational acceleration at a given altitude in m/s^2

    g = G_0 * (R_EARTH/(R_EARTH+altitude))**2

    return g
```



```

def drag(rho, velocity):
    # Function returns drag force in Newtons

    d = 0.5 * C_DRAG * rho * velocity**2 * (np.pi * (DIAMETER/2)**2)

    return d

def pitch_angle(time, velocity):
    # Function returns the pitch angle in radians at a given time and velocity vector (x, z)

    # Resolve components
    vx = velocity[0]
    vz = velocity[1]

    if time <= 10:
        return ANGLE_INITIAL * np.pi/180

    elif time <= 20:
        return PITCH_OVER_ANGLE * np.pi/180

    else:
        return np.arctan2(vz, vx) # Use arctan2 to avoid division by zero (vectorised arctan)

def mass(time):
    # Function returns the mass of the rocket at a given time, noting that burning stops after BURN_TIME

    if time <= BURN_TIME:
        return M_0 - (ENGINE_THRUST / (SPECIFIC_IMPULSE * G_0)) * time
    else:
        return M_0 - (ENGINE_THRUST / (SPECIFIC_IMPULSE * G_0)) * BURN_TIME

def thrust(time):
    # Function returns the thrust of the rocket at a given time, noting that burning stops after BURN_TIME

    if time <= BURN_TIME:
        return ENGINE_THRUST
    else:
        return 0

def derivatives(time, state):
    # Function calculates the derivatives of the state vector at a given time

    # Get each element of state vector
    x, z, vx, vz = state

    speed = np.sqrt(vx**2 + vz**2)

    # Calculate required quantities at this state
    rho = density(z)
    m = mass(time)
    T = thrust(time)
    D = drag(rho, speed)
    phi = pitch_angle(time, [vx, vz])
    g = gravity(z)
    angle = np.arctan2(vz, vx) # Angle of velocity vector, for drag direction

```

```

# Calculate derivatives
dx_dt = vx
dz_dt = vz
dvx_dt = (T * np.cos(phi) - D * np.cos(angle)) / m
dvz_dt = (T * np.sin(phi) - D * np.sin(angle)) / m - g

# Return vector qdot
return [dx_dt, dz_dt, dvx_dt, dvz_dt]

# Termination events. Hit_ground is required for assignment to stop it 1 minute early, but I added zero
def zero_mass(time, state):
    return mass(time) # We want to stop if mass is zero
zero_mass.terminal = True # Tells the integrator to stop when this event occurs
zero_mass.direction = -1 # We want to detect only when mass is decreasing

def hit_ground(time, state):
    return state[1] # We want to stop when altitude (z) is zero
hit_ground.terminal = True # Tells the integrator to stop when this event occurs
hit_ground.direction = -1 # We want to detect only when z is decreasing

# -----
# Main solver
# Uses Runge-Kutta 4(5) method (RK45) in the scipy solve_ivp function
# Note: assignment says to use scipy.integrate.ode, but this is deprecated in favour of solve_ivp
# -----
solution = solve_ivp(
    fun = derivatives, # Define derivative function to pass into integrator
    t_span = [0, SIMULATION_TIME], # Start and end times
    y0 = state0, # Initial state
    t_eval = np.arange(0, SIMULATION_TIME, TIME_STEP), # Times at which to store the computed solution
    rtol = 1e-10, # Desired relative error
    atol = 1e-10, # Desired absolute error
    method = 'RK45', # Runge-Kutta 4(5) method
    dense_output = True, # Allow interpolation of solution (not needed for task, but useful to have)
    events = [zero_mass, hit_ground], # Stop integration when rocket hits the ground or mass is zero
)

# Store results
times = solution.t
states = solution.y.T

# End simulation 1 minute before impact if it hits the ground
if max(times) < SIMULATION_TIME - TIME_STEP:
    downrange_collision = states[-1,0] # x position at impact
    if mass(max(times) + TIME_STEP) <= 0:
        print(f"Rocket ran out of fuel at t = {max(times):.2f} secs, {downrange_collision/1000:.2f} km")
    else:
        print(f"Rocket hit the ground at t = {max(times):.2f} secs, {downrange_collision/1000:.2f} km")

end_time = max(times) - 60 # 1 minute before impact

if max(times) < 5 * 60: # Chosen 5 minutes as a threshold for "too early" so it looks better
    print("Rocket impacted too early, so showing full trajectory.")
else:
    times = times[times <= end_time] # Keep only times before end_time
    states = states[:len(times), :] # Keep only corresponding states

```

```

# Unpack states
x = states[:,0]
z = states[:,1]
vx = states[:,2]
vz = states[:,3]

# Calculate additional quantities
speed = np.sqrt(vx**2 + vz**2) # Speed at each time step
mass_values = np.array([mass(t) for t in times]) # Calculate mass at each time step
phi_values = np.array([pitch_angle(t, [vx[i], vz[i]]) for i, t in enumerate(times)]) # Calculate pitch angle at each time step
thrust_values = np.array([thrust(t) for t in times]) # Calculate thrust at each time step
drag_values = np.array([drag(density(z[i]), speed[i]) for i in range(len(times))]) # Calculate drag at each time step

# Print required results
max_altitude = max(z) / 1000 # Convert to km
max_speed = max(speed) / 1000 # Convert to km/s
max_drag = max(drag_values) / 1000 # Convert to kN

# Get the times that these events occur to plot them on graphs
max_altitude_time = times[np.argmax(z)]
max_speed_time = times[np.argmax(speed)]
max_drag_time = times[np.argmax(drag_values)]

print(f"Maximum Altitude (Apogee): {max_altitude:.2f} km at t = {max_altitude_time:.2f} secs")
print(f"Maximum Speed: {max_speed:.2f} km/s at t = {max_speed_time:.2f} secs")
print(f"Maximum Drag (Max-Q): {max_drag:.2f} kN at t = {max_drag_time:.2f} secs")

# -----
# Produce required plots
# -----

# Coordinate plot (x-z plane) (Unassessed)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12,6))
ax1.plot(x/1000, z/1000) # Convert to km
ax1.set_xlabel('Downrange Distance (km)')
ax1.set_ylabel('Altitude (km)')
ax1.set_title('Equal Axes')
ax1.axis('equal')
ax1.grid()
ax2.plot(x/1000, z/1000) # Convert to km
ax2.set_xlim(left=0) # Zoom in
ax2.set_ylim(bottom=0)
ax2.set_xlabel('Downrange Distance (km)')
ax2.set_ylabel('Altitude (km)')
ax2.set_title('Scaled')
ax2.grid()
fig.suptitle('Rocket Trajectory (x-z Plane) (UNASSESSED)')
plt.tight_layout()
plt.savefig('trajectory.png')

# Altitude vs Time plot (Assessed)
plt.figure()
plt.plot(times, z/1000) # Convert to km
plt.axvline(x=BURN_TIME, color='red', linestyle='--', label="Burnout") # Burnout line
plt.axvline(x=max_altitude_time, color='k', linestyle='--', label="Max altitude") # Max altitude line

```

```

plt.xlabel('Time (s)')
plt.ylabel('Altitude (km)')
plt.title('Rocket Altitude vs Time')
plt.xlim(left=0)
plt.ylim(bottom=0)
plt.grid()
plt.legend()
plt.savefig('altitude_vs_time.png')

# Downrange Distance vs Time plot (Assessed)
plt.figure()
plt.plot(times, x/1000) # Convert to km
plt.axvline(x=BURN_TIME, color='red', linestyle='--', label="Burnout") # Burnout line
plt.xlabel('Time (s)')
plt.ylabel('Downrange Distance (km)')
plt.title('Rocket Downrange Distance vs Time')
plt.xlim(left=0)
plt.ylim(bottom=0)
plt.grid()
plt.legend()
plt.savefig('downrange_distance_vs_time.png')

# Vertical Velocity vs Time plot (Assessed)
plt.figure()
plt.plot(times, vz)
plt.axvline(x=BURN_TIME, color='red', linestyle='--', label="Burnout") # Burnout line
plt.axvline(x=max_altitude_time, color='k', linestyle='--', label="Max altitude") # Max altitude line
plt.xlabel('Time (s)')
plt.ylabel('Vertical Velocity (m/s)')
plt.title('Rocket Vertical Velocity vs Time')
plt.xlim(left=0)
plt.grid()
plt.legend()
plt.savefig('vertical_velocity_vs_time.png')

# Horizontal Velocity vs Time plot (Assessed)
plt.figure()
plt.plot(times, vx)
plt.axvline(x=BURN_TIME, color='red', linestyle='--', label="Burnout") # Burnout line
plt.xlabel('Time (s)')
plt.ylabel('Horizontal Velocity (m/s)')
plt.title('Rocket Horizontal Velocity vs Time')
plt.xlim(left=0)
plt.grid()
plt.legend()
plt.savefig('horizontal_velocity_vs_time.png')

# Pitch angle vs Time plot (Assessed)
plt.figure()
plt.plot(times, phi_values * 180/np.pi) # Convert to degrees
plt.axvline(x=BURN_TIME, color='red', linestyle='--', label="Burnout") # Burnout line
plt.axvline(x=max_altitude_time, color='k', linestyle='--', label="Max altitude") # Max altitude line
plt.xlabel('Time (s)')
plt.ylabel('Pitch Angle (degrees)')
plt.title('Rocket Pitch Angle vs Time')
plt.xlim(left=0)

```

```

plt.grid()
plt.legend()
plt.savefig('pitch_angle_vs_time.png')

# Mass vs Time plot (Assessed)
plt.figure()
plt.plot(times, mass_values)
plt.axvline(x=BURN_TIME, color='red', linestyle='--', label="Burnout") # Burnout line
plt.xlabel('Time (s)')
plt.ylabel('Rocket Mass (kg)')
plt.title('Rocket Mass vs Time')
plt.xlim(left=0)
plt.ylim(bottom=0)
plt.grid()
plt.legend()
plt.savefig('mass_vs_time.png')

# Speed vs Time plot (Unassessed)
plt.figure()
plt.plot(times, speed)
plt.axvline(x=BURN_TIME, color='red', linestyle='--', label="Burnout") # Burnout line
plt.axvline(x=max_altitude_time, color='k', linestyle='--', label="Max altitude") # Max altitude line
plt.xlabel('Time (s)')
plt.ylabel('Rocket Speed (m/s)')
plt.title('Rocket Speed vs Time')
plt.xlim(left=0)
plt.grid()
plt.legend()
plt.savefig('speed_vs_time.png')

# Thrust vs Time plot (Unassessed)
plt.figure()
plt.plot(times, thrust_values/1000) # Convert to kN
plt.axvline(x=BURN_TIME, color='red', linestyle='--', label="Burnout") # Burnout line
plt.xlabel('Time (s)')
plt.ylabel('Rocket Thrust (kN)')
plt.title('Rocket Thrust vs Time')
plt.xlim(left=0)
plt.ylim(bottom=0)
plt.grid()
plt.legend()
plt.savefig('thrust_vs_time.png')

# Drag vs Time plot (Unassessed)
plt.figure()
plt.plot(times, drag_values/1000) # Convert to kN
plt.axvline(x=max_drag_time, color='k', linestyle='--', label="Max-Q") # Max-Q line
plt.axvline(x=BURN_TIME, color='red', linestyle='--', label="Burnout") # Burnout line
plt.xlabel('Time (s)')
plt.ylabel('Rocket Drag (kN)')
plt.title('Rocket Drag vs Time')
plt.xlim(left=0)
plt.ylim(bottom=0)
plt.grid()
plt.legend()
plt.savefig('drag_vs_time.png')

```